

计算机系统基础实验报告

Lab 0: GitLab

姓名：陈璋睿 学号：23300160006

2026 年 9 月 30 日

一、实验目的与任务

本次实验通过修改 C 程序、记录提交以及合并分支，学习 Git 的基本工作流程。我需要理解工作区、暂存区与本地仓库的关系，掌握分支查看和切换方法，并在真实的合并冲突中判断应该保留的代码。

依据仓库 README 的要求，本报告回答文档中的三个问题，概括两篇阅读材料，并记录代码修改、分支提交、冲突处理和验证结果。

二、实验环境与初始修改

2.1 实验环境

项目	实际环境
操作系统	Ubuntu 22.04.5 LTS
版本控制工具	Git 2.34.1
C 编译器	GCC 11.4.0
编辑工具	VS Code（代码编辑及合并编辑器）
实验仓库	ICS_lab0，主分支 main，实验分支 feature
个人仓库	https://github.com/ritachen3656251-design/ICS_lab0

2.2 建立个人仓库并修改 main.c

按照课程说明，使用模板建立个人仓库并将其克隆到本地。在 main.c 的 TODO 位置填写输出语句，然后完成第一次修改提交。仓库历史中的对应提交为 b407cb5，提交说明为 style:change the sentence。

```
printf("Hello, welcome to FDU! \n");
```

后续分支实验继续在这一版本上增加 new_number 函数及调用代码，在两个分支分别实现加法和乘法，再将它们合并。

三、文档问题回答

3.1 你之前有过多人协同开发的经历吗？如何分工协作？

我有过非正式的多人课程项目开发经历。当时虽然使用了 GitHub 仓库分享初始代码，也把组员添加为协作者，但并没有真正使用 Git 的分支与合并流程。大家克隆代码后分别在本地运行和修改，最后通过压缩包传递完整代码，再人工整合。

这种方式的问题是，任务的接口和修改边界不够明确，彼此的函数修改很容易互相影响。整合时难以分清哪些内容是新增的，哪些改动覆盖了其他人的工作，因此返工较多。以后我希望先约定模块接口，再为独立任务建立分支，通过清晰的提交记录和合并审查整合成果。

3.2 Git 为什么要设计“暂存—提交”两个步骤？

工作区保存正在编辑的文件，暂存区保存准备纳入下一次提交的内容，而提交将暂存区的快照记录到仓库历史中。将这两个步骤分开，使当前正在做的修改和这一次要记录的修改可以不同。

首先，暂存区允许按文件或代码片段选择内容。例如，同一个文件中同时修改了功能和注释，可以使用 `git add -p` 选择片段，分别形成逻辑清晰的提交。这样查看历史、定位错误和撤销某项改动都会更容易。

其次，暂存后可以用 `git diff --cached` 检查将要提交的内容，减少误提交无关修改的情况。但暂存区不会自动检查或消除 bug，仍然需要人工审查和测试。暂存以后再次编辑文件，也需要重新暂存，新的编辑才会进入下一次普通提交。

最后，遇到紧急修复时，可以只暂存修复所需的改动，保留尚未完成的开发内容。不过，工作区里未暂存的内容没有因此获得提交历史的保护。若需要切换到独立修复环境，还可以配合 `stash` 或 `worktree`。

3.3 `git branch` 和 `git branch -a` 有什么区别？

根据 Git 官方文档，在不指定分支名的列举用法下，`git branch` 显示本地分支，`git branch -a` 同时显示本地分支和远程跟踪分支。当前所在分支通常以星号标记。`git branch -r` 则仅列出远程跟踪分支。

本仓库的实际输出可概括为：

```
$ git branch
  feature
* main
$ git branch -a
  feature
* main
remotes/origin/HEAD -> origin/main
remotes/origin/main
```

远程跟踪分支是本地保存的远程分支状态引用，例如 `origin/main`，通常由 `fetch` 更新。实际开发一般在本地分支上提交，再通过 `push` 更新远程仓库。

四、阅读材料与学习体会

4.1 Commit Message 规范

《Commit message 和 Change log 编写指南》介绍了以类型、影响范围和简短描述组织提交标题的方法。正文解释修改原因，页脚可记录不兼容变更和关联问题。统一格式有助于阅读历史、筛选改动和生成更新日志，文章也介绍了辅助填写与校验的工具。

我的理解是，提交信息应当让后来阅读代码的人知道这次为什么改。本次实验中的功能提交分别记录了加法、乘法，合并提交说明了冲突解决，比笼统地写“修改代码”更有助于定位实验步骤。以后应进一步统一空格、拼写和描述方式。

4.2 Git Flow 分支控制

《Gitflow 使用规范》将发布、开发和修复放在不同分支：master 保存发布版本，develop 汇集开发成果，feature 开发新功能，release 用于发布前验证，hotfix 处理已发布版本的紧急问题。完成后，将相关修改合并到需要同步的分支，并通过标签标识发布版本。

这让我理解了分支的作用，不同任务可以在明确的边界内推进，再按照约定整合，不过本次实验没有实施完整的 Gitflow，只做了简单的分支管理。

4.3 为什么要学习 Git？

Git 使代码修改可以追溯。出现问题时，我可以检查某次提交改变了哪些文件、修改的原因是什么，并比较问题出现前后的版本。相比保存多个压缩包，提交历史提供了更清楚的演变过程。

Git 也支持多人并行协作。独立分支能够隔离尚未完成的工作，合并负责整合成果。冲突提示说明某些内容需要开发者判断。本次加法和乘法的选择，就不能只靠工具自动决定。清晰的分工、代码审查和测试仍然是必要的。

五、分支修改与提交

5.1 在 feature 分支实现加法

从共同版本 b407cb5 建立 feature 分支，增加 new_number 函数，返回两参数之和。在主函数中令 a=10、b=20，调用函数并输出结果。功能提交为 20abe2b (feat:add a func(addition))，随后有格式调整提交 e7bd0f0。

```
ICS_lab0 > C main.c
1  #include <stdio.h>
2
3  int new_number(int num1, int num2){
4      return num1 + num2;
5  }
6
7  int main()
8  {
9      // @TODO: print a sentence you want.
10     printf("Hello, welcome to FDU!\n");
11     int a = 10;
12     int b = 20;
13     int num = new_number(a, b);
14     printf("新的数字是: %d", num);
15     return 0;
16 }
17
```

图 1: feature 分支的加法实现

5.2 在 main 分支实现乘法

切换回 main 分支，在相同位置增加同名函数，但使其返回两个参数的乘积。功能提交为 c2885d8 (feat:add the func(multiply))，随后有格式调整提交 fe80286。两个分支至此各有自己的提交。

```
ICS_lab0 > C main.c
1  #include <stdio.h>
2
3  int new_number(int num1, int num2){
4      return num1 * num2;
5  }
6  int main()
7  {
8      // @TODO: print a sentence you want.
9      printf("Hello, welcome to FDU!\n");
10     int a = 10;
11     int b = 20;
12     int num = new_number(a, b);
13     printf("新的数字是: %d", num);
14     return 0;
15 }
```

图 2: main 分支的乘法实现

共同版本尚未包含该函数，两个分支在同一位置加入了不一致的实现。加法与乘法不能作为同一个函数的最终定义同时保留，这为后续合并提供了实际冲突。

六、合并冲突的出现与处理

6.1 合并与冲突定位

切换回 main 后执行 `git merge feature`。合并暂停后，在编辑器中可看到冲突标记：<<<<<<< HEAD 以下是当前 main 的内容，=====
分隔双方内容，>>>>>>> feature 结束传入分支的内容。

```
#include <stdio.h>

采用当前更改 | 采用传入的更改 | 保留双方更改 | 比较变更
<<<<<< HEAD (当前更改)
int new_number(int num1, int num2){
    return num1 + num2;
}

=====
int new_number(int num1, int num2){
    return num1 + num2;
}

>>>>>> feature (传入的更改)
int main()
{
    // @T000: print a sentence you want.
    printf("Hello, welcome to FDU!\n");
    int a = 10;
    int b = 20;
    int num = new_number(a, b);
采用当前更改 | 采用传入的更改 | 保留双方更改 | 比较变更
<<<<<< HEAD (当前更改)
    printf("新的数字是: %d", num);
    return 0;
}

=====
    printf("新的数字是: %d", num);
    return 0;
}

>>>>>> feature (传入的更改)
```

图 3: main.c 中实际出现的冲突标记

6.2 比较两侧内容并确定结果

打开合并编辑器，对比 feature 的加法与 main 的乘法。本次决定保留加法实现，并保留主函数中的参数、调用和输出语句。图 4 是编辑过程中的状态：结果区域曾出现两个同名函数，且仍有未处理的冲突，所以它不是最终代码，不能直接作为完成合并的依据。

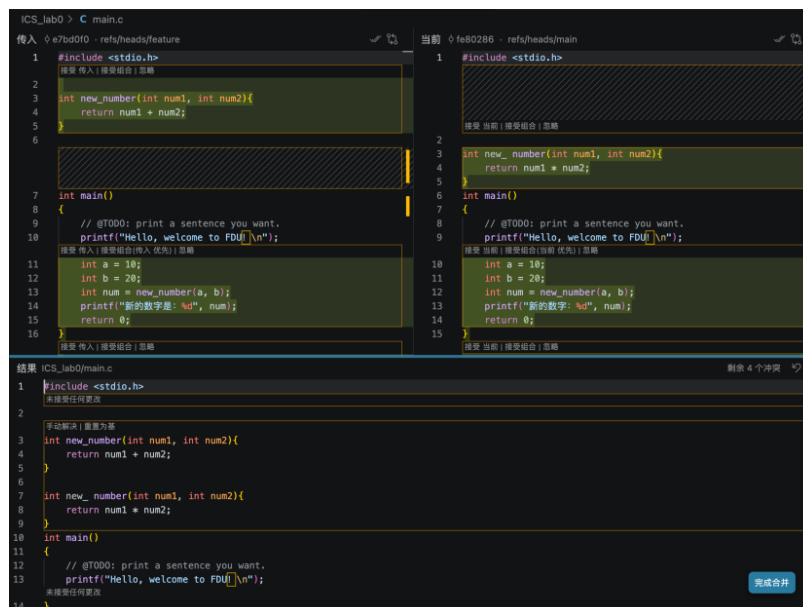


图 4: 合并编辑器中的中间状态，随后继续整理代码

七、最终结果与验证

7.1 冲突解决后的代码及合并提交

完成整理后，删除冲突标记及重复定义，只保留一个返回 `num1 + num2` 的函数。最终代码如图 5 所示。

```
ICS_lab0 > C main.c
1  #include <stdio.h>
2
3  int new_number(int num1, int num2){
4      return num1 + num2;
5  }
6
7  int main()
8  {
9      // @TODO: print a sentence you want.
10     printf("Hello, welcome to FDU!\n");
11     int a = 10;
12     int b = 20;
13     int num = new_number(a, b);
14     printf("新的数字是: %d", num);
15     return 0;
16 }
```

图 5：冲突解决后的 main.c，最终选择加法实现

保存文件后，在原实验过程中暂存并提交，生成合并提交 8a1b892，说明为 `merge: resolve main.c conflict`。图 6 是该次历史操作的截图。

```
(base) root@dsw-487972-6f7dfc874b-c9j2f:/cpfs/user/zhangpingrui/code_czr/ICS_lab0# git add .
(base) root@dsw-487972-6f7dfc874b-c9j2f:/cpfs/user/zhangpingrui/code_czr/ICS_lab0# git commit -m "merge: resolve main.c conflict"
[main 8a1b892] merge: resolve main.c conflict
```

图 6：原实验中完成合并提交的终端截图

7.2 提交历史与运行结果

本地历史核对显示，8a1b892 的两个父提交分别为 fe80286 (main) 与 e7bd0f0 (feature)，证明两条分支的历史已汇合。之后的 94e93b1 为 README 更新提交，当前所在分支为 main。

整理报告时对最终源码进行了补充编译验证，使用 `gcc -Wall -Wextra` 编译到临时目录，编译成功且无警告，运行结果为：

```
Hello, welcome to FDU!
新的数字是: 30
```

结果中的 30 与选择保留的加法实现一致。

7.3 对冲突原因的理解

两个分支修改同一个文件并不一定产生冲突。修改互不重叠时通常可以自动合并。在同一位置加入不同内容、对重叠区域作不兼容修改，或一方删除而另一方修改时，可能需要手动处理。本次主要冲突来自同一位置的加法与乘法函数定义，截图中还包含主函数末尾区域的冲突。